



hexagone

École Supérieure d'Informatique

Fichiers et données

4PYTH – Langage Python

Chris CHEVALIER



hexagone

École Supérieure d'Informatique

Sommaire

- #1 Lire et écrire des fichiers
- #2 JSON et sauvegarde d'objets



#1

Lire et écrire des fichiers

Fichiers et données



Ouvrir un fichier

```
notes.py

with open("notes.txt", "w", encoding="utf-8") as file:
    file.write("Audit du serveur srv-web\n")

with open("notes.txt", encoding="utf-8") as file:
    content = file.read()
print(content)    # Audit du serveur srv-web
```

Mode	Effet
"r"	lecture (par défaut)
"w"	écriture, qui écrase le fichier
"a"	ajout à la fin

D'autres modes existent, comme "r+" (lecture et écriture). Ils sont décrits dans la [documentation de open\(\)](#).

Précisez toujours `encoding="utf-8"`. Sans lui, Windows utilise un autre encodage et les accents deviennent illisibles.



Lire ligne par ligne

```
auth.log
Oct  9 10:02:11 srv-web sshd[2210]: Failed password for root from 203.0.113.45 port 51122 ssh2
Oct  9 10:15:44 srv-web sshd[2303]: Failed password for root from 198.51.100.23 port 40031 ssh2
Oct  9 11:31:02 srv-web sshd[2410]: Accepted password for Cosmo from 192.0.2.10 port 42230 ssh2
```

```
analysis.py
failures = {}
with open("auth.log", encoding="utf-8") as log:
    for line in log:
        if "Failed password" in line:
            words = line.split()
            ip = words[words.index("from") + 1]
            failures[ip] = failures.get(ip, 0) + 1
print(failures)
# {'203.0.113.45': 1, '198.51.100.23': 1}
```

- `for line in log` lit **une ligne à la fois**
- La mémoire reste stable, même pour un journal de plusieurs gigaoctets



Écrire et ajouter

```
event_log.py

from datetime import datetime  # dates et heures

def log_event(message):
    # date et heure actuelles, au format 2026-10-09T10:42:05
    now = datetime.now().isoformat(timespec="seconds")
    with open("actions.log", "a", encoding="utf-8") as file:
        file.write(f"{now} {message}\n")

log_event("Ajout de Jagger Fount")
log_event("Retrait de Hugo Crast")
```

```
actions.log

2026-10-09T10:42:05 Ajout de Jagger Fount
2026-10-09T10:42:05 Retrait de Hugo Crast
```

- Le module standard `datetime` gère les dates et les heures
- Le mode `"a"` **ajoute** à la fin du journal
- `write()` n'ajoute pas de retour à la ligne. Écrivez le `\n` vous-même



Les erreurs de fichier

```
reading.py

try:
    with open("report.txt", encoding="utf-8") as file:
        print(file.read())
except FileNotFoundError:
    print("Fichier introuvable")
except PermissionError:
    print("Accès refusé : droits insuffisants")
```

```
paths.py

from pathlib import Path

path = Path("auth.log")
print(path.exists())    # True
print(path.suffix)      # .log
```

- Un fichier peut manquer ou être protégé. Le programme ne doit pas planter
- `pathlib` manipule les chemins sous Windows, macOS et Linux



#2 JSON et sauvegarde d'objets

Fichiers et données



Le format JSON

- **JSON** (*JavaScript Object Notation*) est un format texte de données structurées, lisible par les humains
- C'est le format des API web, des fichiers de configuration... et des sauvegardes de nombreux logiciels

JSON	Python
objet {}	dict
tableau []	list
chaîne, nombre	str, int, float
true / false / null	True / False / None

```
config.json
{
  "server": "srv-web",
  "ports": [22, 443],
  "mfa": true,
  "owner": "Équipe SOC"
}
```



Écrire et lire du JSON

```
config.py

import json

config = {"server": "srv-web", "ports": [22, 443],
          "mfa": True, "owner": "Équipe SOC"}

with open("config.json", "w", encoding="utf-8") as file:
    json.dump(config, file, indent=2, ensure_ascii=False)

with open("config.json", encoding="utf-8") as file:
    loaded = json.load(file)
    print(loaded["ports"])           # [22, 443]

text = json.dumps(config)           # dict → str
print(json.loads(text)["mfa"])      # True
```

- `dump` / `load` : vers ou depuis un **fichier**
- `dumps` / `loads` : vers ou depuis une **chaîne** (le « s » de *string*)
- `indent=2` : un fichier indenté, donc lisible
- `ensure_ascii=False` : les accents restent lisibles



Sauvegarder des objets

```
user.py

class User:
    def __init__(self, username, role):
        self.username = username
        self.role = role

    def to_dict(self):
        return {"username": self.username, "role": self.role}

def user_from_dict(data):
    return User(data["username"], data["role"])
```

```
main.py

data = []
for user in users:
    data.append(user.to_dict())
with open("users.json", "w", encoding="utf-8") as file:
    json.dump(data, file, indent=2)
```

- JSON ne connaît pas vos classes.
`json.dumps(users)` lève
`TypeError: Object of type User is not JSON serializable`
- On convertit donc chaque objet en **dictionnaire**
- Au chargement,
`user_from_dict()` refait l'objet
- Chaque classe a son propre `to_dict()`



Charger sans planter

```
save.py

def load_users(path):
    try:
        with open(path, encoding="utf-8") as file:
            data = json.load(file)
    except FileNotFoundError:
        return [] # premier lancement
    except json.JSONDecodeError:
        print("Sauvegarde corrompue")
        return []
    users = []
    for item in data:
        users.append(user_from_dict(item))
    return users

print(len(load_users("absent.json"))) # 0
print(len(load_users("users.json"))) # 2
```

- Au **premier lancement**, la sauvegarde n'existe pas encore. Ce n'est pas une erreur
- Un fichier tronqué ou modifié à la main lève l'erreur `json.JSONDecodeError`
- Dans les deux cas, le programme démarre avec des données vides



Côté sécurité

```
user.py

ROLES = ["analyste", "auditeur", "admin"]

def user_from_dict(data):
    if data.get("role") not in ROLES:
        raise ValueError(f"rôle inconnu : {data.get('role')}")
    return User(data["username"], data["role"])

user_from_dict({"username": "eve", "role": "root"})
# ValueError: rôle inconnu : root
```

- `json` ne fait que lire des données. Il n'exécute jamais de code
- Un fichier JSON se modifie pourtant à la main. Les données chargées **se valident** donc, par liste blanche

Jamais d' `eval()` (qui exécute une chaîne comme du code Python) sur des données non fiables. Un fichier piégé suffirait à **exécuter du code** sur votre machine.



Récapitulatif

Notion	À retenir
Fichiers texte	<code>with open(..., encoding="utf-8")</code> ; modes "r" , "w" et "a" ; lecture ligne par ligne
Dates	<code>datetime.now()</code> pour horodater un journal
Erreurs de fichier	<code>FileNotFoundError</code> , <code>PermissionError</code>
JSON	<code>json.dump()</code> et <code>json.load()</code> ; <code>to_dict()</code> pour les objets ; chargement qui ne plante pas
Sécurité	jamais d' <code>eval()</code> sur des données non fiables ; valider les données chargées



Cet après-midi

Horaire	Programme
14h00 – 16h45	TP1 – Fondation : gestion de l'équipage, avec mon accompagnement. Ceux qui ont fini enchaînent sur le TP2
16h45 – 17h00	Kahoot : 10 questions sur la journée

Le **défi du jour 2** est **facultatif** et non noté. Il s'adresse à ceux qui ont terminé le TP1.



Des questions ?

Chris CHEVALIER

chris.chevalier@ecole-hexagone.com